Week 37, Tuesday Session

Flipped classroom: your questions, then gradient descent in practice

Bring your laptop – we work in `week37tuesday.ipynb`

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

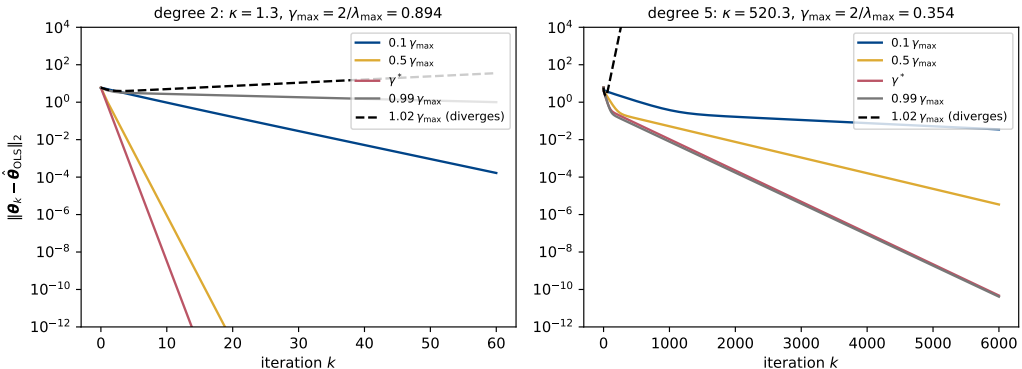
How today works

The format (as every Tuesday)

Mondays are lectures; Tuesdays belong to your questions and to concrete cases. You have heard the lecture and skimmed Sections 3.14, 4.1–4.6 and 4.14.

1. **First ∼20 min:** your open questions from Monday's lecture and from the reading, discussed in plenum. Cross-validation questions are welcome here too – Project 1 part d) is still open.
2. **Rest of the double session:** *two* cases only – your own gradient descent code for OLS and Ridge with analytical gradients, and momentum plus the automatic gradient on the Runge function – worked in pairs or small groups from `week37tuesday.ipynb`. Deep beats broad: the goal is that you leave with a working, *checked* gradient descent code and an understanding of what the learning rate can and cannot be. Case 1 is this week's exercises; Cases 1 and 2 together are parts e) and f) of Project 1.
3. **Last 5 min:** wrap-up, exercise deadline, and what next Monday builds on.

Picking up from Monday: the learning rate is a property of the data



Monday's central result, Eq. (4.19): along each eigendirection of the Hessian the error contracts by $|1 - \gamma\lambda_i|$ per step, so the iteration is stable iff $\gamma < 2/\lambda_{\max}$ and needs $\propto \kappa$ iterations. **Left**: degree 2, $\kappa = 1.3$, 11 iterations at γ^*. **Right**: degree 5 on the same data, $\kappa = 520$, 4964 iterations at γ^* – and $\gamma_{\max}$ has moved from 0.89 to 0.35. Today you build the code that produced this figure, check the bound on *your* Hessian, and then make the slow case fast with momentum.

Case 1: your own gradient descent for OLS and Ridge, checked

Goal: exercises 1–4 of this week, done together (notebook, Case 1; seed 2026 gives exactly the lecture numbers). Data $f(x) = 2 - x + 5x^2$ on $[-2, 2]$, $n = 100$, noise $\sigma = 0.5$.

1. **Scale** (exercise 1): standardise the columns of the polynomial design matrix, centre y, no intercept column. Why does centring y let us drop the intercept?
2. **Gradients and Hessian** (exercise 2): write `gradient(theta, X, y, lam)` from Eqs. (4.13) and (4.17), and `hessian_eigs`. Check the gradient at a random $\boldsymbol{\theta}$ against a central difference, $h = 10^{-5}$ (agreement $\sim 10^{-9}$, not 10^{-15}: Case 2 says why).
3. **Closed forms** (exercise 3): OLS and Ridge with the $n\lambda$I of Eq. (3.95). Compare with `Ridge(alpha=n*lam)`.
4. **Gradient descent** (exercise 4): the loop of Eq. (4.15) with a stopping criterion on $\|\nabla C\|$. Degree 2: run $\gamma \in \{0.01, 0.1, 0.5, 0.8\}$; then degree 5 with the same values. What happens at 0.5 and 0.8 for degree 5, and why?
5. **The bound**: compute $\lambda_{\max}, \lambda_{\min}$ of the Hessian, hence $\gamma_{\max} = 2/\lambda_{\max}$ and $\gamma^* = 2/(\lambda_{\max} + \lambda_{\min})$; run at $0.99\,\gamma_{\max}$ and $1.02\,\gamma_{\max}$. Then Ridge with $\lambda \in \{10^{-2}, 10^{-1}\}$: what does the penalty do to κ and to the iteration count?

Checkpoints (plenary after ∼30 min)

Degree 2: $\hat{\boldsymbol{\theta}} = (-1.077,\ 5.855)$, $\kappa = 1.3$, $\gamma^* = 0.5$, 11 iterations. Degree 5: $\kappa = 520$, $\gamma_{\max} = 0.354$, γ^* needs 4964 iterations, $\gamma = 0.1$ needs 11 836; Ridge $\lambda = 0.1$ brings κ to 28.

Case 1: the code you should end up with

```
def gradient(theta, X, y, lam=0.0):
    """Eqs. (4.13) and (4.17): gradient of (1/n)||X theta - y||^2 + lam theta^T theta."""
    n = len(y)
    return (2.0 / n) * X.T @ (X @ theta - y) + 2.0 * lam * theta

def gradient_descent(X, y, gamma, lam=0.0, num_iters=1000, tol=1e-8, theta0=None):
    """Plain gradient descent, Eq. (4.15). Returns the iterates and the number of steps."""
    p = X.shape[1]
    theta = np.zeros(p) if theta0 is None else theta0.copy()
    history = [theta.copy()]
    for t in range(num_iters):
        g = gradient(theta, X, y, lam)
        theta = theta - gamma * g
        history.append(theta.copy())
        if np.linalg.norm(g) < tol:
            break
    return np.array(history), t + 1

def hessian_eigs(X, lam=0.0):
    """Eigenvalues of the Hessian (2/n) X^T X + 2 lambda I, Eqs. (4.14) and (4.17)."""
    n = len(X)
    return np.linalg.eigvalsh((2.0 / n) * X.T @ X + 2.0 * lam * np.eye(X.shape[1]))
```

Three functions, each doing one thing: `gradient` is the only place the cost enters; `gradient_descent` is joined by `momentum_descent` today and by the adaptive methods next week. Keep the `history`: it is what you plot.

Case 1: do you have the insight?

1. Your loop stopped after its 1000 allowed iterations with $\|\nabla C\| = 10^{-4}$, small but not below tolerance. Has it converged? What single number tells you how far off you may still be?
2. Standardising the columns changed the iteration count by two orders of magnitude but not the fitted curve. Why does gradient descent care about the scaling when the closed form does not?
3. Ridge with $\lambda = 0.1$ converged in a twentieth of the iterations OLS needed. Is the point it converged to a better solution of the OLS problem?
4. The same code diverged at $\gamma = 0.5$ on degree 5 and converged at $\gamma = 0.8$ on degree 2. Where does the boundary come from, and what should the code compute before it starts?

Case 1: do you have the insight?

1. Your loop stopped after its 1000 allowed iterations with $\|\nabla C\| = 10^{-4}$, small but not below tolerance. Has it converged? What single number tells you how far off you may still be?
2. Standardising the columns changed the iteration count by two orders of magnitude but not the fitted curve. Why does gradient descent care about the scaling when the closed form does not?
3. Ridge with $\lambda = 0.1$ converged in a twentieth of the iterations OLS needed. Is the point it converged to a better solution of the OLS problem?
4. The same code diverged at $\gamma = 0.5$ on degree 5 and converged at $\gamma = 0.8$ on degree 2. Where does the boundary come from, and what should the code compute before it starts?

What we hope you say

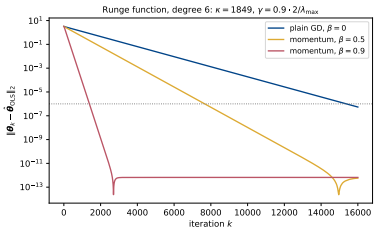
(1) No. The slow direction contracts by $|1 - \gamma\lambda_{\min}|$ per step; the residual error is $\|\nabla C\|/\lambda_{\min}$ at worst – with $\lambda_{\min} = 0.011$ that is 10^{-2} in $\boldsymbol{\theta}$, the second decimal. Stop on the gradient, not on a count. (2) The closed form solves the normal equations in one go, whatever the units; gradient descent takes the *same* step in every direction, and the columns' scales set the eigenvalue spread κ, Eq. (4.22), hence the iteration count. Same minimum, different road. (3) No – it is the exact solution of a *different* problem, Eq. (4.16). The penalty lifts $\lambda_{\min}$, Eq. (4.17), so the road gets shorter, but the destination moved: bias for variance, Section 3.8. (4) $\gamma < 2/\lambda_{\max}$, Eq. (4.20); $\lambda_{\max}$ is a property of X, not of the algorithm. One line: `eigvalsh(2/n X.T@X).max()`.

Case 2: momentum and the automatic gradient on the Runge function

Goal: parts e) and f) of Project 1 in miniature (notebook, Case 2). Data $f(x) = 1/(1+25x^2)$ on $[-1,1]$, $n = 100$, $\sigma = 0.1$, degree 6, standardised columns, centred y.

1. **The gradient without deriving it:** write the OLS and Ridge costs as ordinary functions of $\boldsymbol{\theta}$, obtain `jax.grad` of each, and compare with your `gradient` from Case 1 at a random $\boldsymbol{\theta}$. Agreement to 10^{-15} – and only to 10^{-6} if you forget `jax_enable_x64`.
2. **Finite differences against AD:** the error of the central difference of one component against h from 10^{-13} to 10^{-1} (reproduce panel (b) of `week37_ad_check.pdf`). Then the same for $f(x) = \sin(2\pi x + x^2)$ at $x = 1$ (panel (a)). Why do the two panels differ on the right?
3. **Momentum:** add `momentum_descent`, Eq. (4.23) – one line more than Case 1. At $\gamma = 0.9 \cdot 2/\lambda_{\max}$ run $\beta \in \{0, 0.5, 0.9\}$ and count the iterations to $\|\boldsymbol{\theta}_k - \hat{\boldsymbol{\theta}}\| < 10^{-6}$. Reproduce `week37_momentum.pdf`.
4. **Sensitivity to γ:** repeat step 3 at $\gamma_{\max}/4$ and at $1.5\,\gamma_{\max}$. Does momentum change where the iteration diverges? (Section 4.6: stable iff $\gamma\lambda_{\max} < 2(1+\beta)$.)
5. **Swap the gradient:** run momentum with the JAX gradient instead of the analytical one – the update rule does not care. Then **degree 10**: what is κ, and does anything converge?

Case 2: what you should find



Checkpoints (seed 2026)

- $\texttt{max|AD - hand|} = 1.8 \times 10^{-15}$ for OLS, same order for Ridge.
- Panel (b) has no rising branch: the OLS cost is quadratic, so the central difference has no truncation error and only the $1/h$ rounding error is left. Panel (a) is the general case: best 10^{-11} at $h \approx 10^{-6}$.
- Degree 6: $\kappa = 1849$. Iterations to 10^{-6}: plain 15 374; $\beta = 0.5$: 7 669; $\beta = 0.9$: 1 391 – a factor 11, against the $\sqrt{\kappa} = 43$ ceiling of Eq. (4.26).
- At $1.5\,\gamma_{\max}$ plain gradient descent diverges, $\beta = 0.5$ sits exactly on its stability edge $2(1+\beta)/\lambda_{\max}$ and oscillates for ever, and $\beta = 0.9$ converges in 761 iterations – *faster* than at $0.9\,\gamma_{\max}$.
- Degree 10: $\kappa \approx 2 \times 10^6$; nothing converges in 10^5 iterations, $\beta = 0.9$ included. This is where week 38 – and Newton's method with the JAX Hessian, one step – comes in. **Plenary:** with Ridge, $\lambda = 10^{-3}$, momentum converges at degree 10 in about 3 100 steps (κ drops to 5 000). Is that cheating?

Case 2: do you have the insight?

1. Your JAX gradient and your analytical gradient agree to 10^{-15}. What has that proved – and what has it *not* proved?
2. Momentum with $\beta = 0.9$ converged at $1.5\,\gamma_{\max}$, where plain gradient descent diverged. Does momentum make the learning rate unimportant?
3. You replace the OLS cost by the cost of a neural network with 10^6 parameters and keep `jax.grad` and `momentum_descent`. What changes in the *code*? What changes in the *theory* you relied on today?
4. Why is reverse mode the right mode for that network, and what does it cost that forward mode does not?

Case 2: do you have the insight?

1. Your JAX gradient and your analytical gradient agree to 10^{-15}. What has that proved – and what has it *not* proved?
2. Momentum with $\beta = 0.9$ converged at $1.5\,\gamma_{\max}$, where plain gradient descent diverged. Does momentum make the learning rate unimportant?
3. You replace the OLS cost by the cost of a neural network with 10^6 parameters and keep `jax.grad` and `momentum_descent`. What changes in the *code*? What changes in the *theory* you relied on today?
4. Why is reverse mode the right mode for that network, and what does it cost that forward mode does not?

What we hope you say

(1) That your derivation of Eq. (4.13) matches the cost you *wrote down* – an error in the algebra would show up as 10^{-2}, not 10^{-15}. It cannot tell you the cost itself is the one you meant (a missing $1/n$, a penalised intercept): both gradients would agree, and both would be wrong. (2) No. The stability edge moved from $2/\lambda_{\max}$ to $2(1+\beta)/\lambda_{\max}$, and the best β is tied to κ, Eq. (4.26); $\beta = 0.5$ sat exactly on its edge and never converged. Two knobs instead of one, both data-dependent. (3) Code: the definition of the cost, nothing else – the point of AD and of the three-function structure. Theory: convexity is gone, so "stationary point = global minimum" (Section 4.1) no longer holds and Eq. (4.20) is only local. Empirical good behaviour, not a theorem. (4) One output, 10^6 inputs: reverse mode gives the whole gradient for two or three cost evaluations, forward mode needs 10^6 sweeps (Sections 4.14.4–4.14.5). The price is the tape – memory, not time.

Wrap-up, deadlines, and Monday

- **What you should own now:** a gradient descent code with a stopping criterion that is *checked* three ways – against the closed form, against a finite difference, against `jax.grad`; the rule $\gamma < 2/\lambda_{\max}$ and the habit of computing $\lambda_{\max}$; momentum as one extra line; and one sentence on what automatic differentiation is not (symbolic, numerical) and what it costs (two or three function evaluations, whatever p).
- **Weekly exercises** (`exercisesweek37.ipynb`): Case 1 as a write-up, plus the sparse ten-feature data set of exercise 5 and momentum in exercise 6. Deadline **Friday September 11 at midnight**.
- **Project 1**, parts e) and f), ask for exactly today's machinery on the Runge function; part f) continues next week with AdaGrad, RMSProp and Adam. The code you wrote today is directly reusable – keep the three-function structure.
- **Next Monday (week 38):** stochastic gradient descent and the adaptive learning rates AdaGrad, RMSProp and Adam (Sections 4.7–4.12), applied to Lasso (Project 1, parts g and h) and to logistic regression (Chapter 5).

Before you leave

One line in an email to us: the muddiest point of the week – what should Monday's first ten minutes revisit?